

pattern_push_heterogeneous

January 3, 2026

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from pathlib import Path
import sys

sys.path.insert(0, str(Path(__file__).parent.parent))
from utils import (generate_patterns, setup_write_experiment,
    ↪setup_sleep_experiment,
                    run_cpp, read_parameters, DATA_DIR)

PLOTS_DIR = Path(__file__).parent.parent / "plots"
PLOTS_DIR.mkdir(exist_ok=True)
```

```
[ ]: sns.set_style("darkgrid")
sns.set_context("paper", font_scale=1.5)
plt.rcParams['text.usetex'] = True
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.serif'] = ['Times']
plt.rcParams.update({
    'font.size': 25,
    'axes.labelsize': 30,
    'axes.titlesize': 30,
    'xtick.labelsize': 25,
    'ytick.labelsize': 25,
    'legend.fontsize': 20,
    'figure.titlesize': 20,
    'lines.linewidth': 2.5,
    'axes.linewidth': 1.5,
    'axes.grid': False,
    'font.weight': 'bold'
})
```

```
[ ]: sparsities = [0.40, 0.45, 0.50, 0.55, 0.60]
N = 600
M = len(sparsities)
```

```

# Generate each pattern with its specific sparsity (sparsity = fraction of
↳ inactive units, P(0) convention)
# Matches generate_patterns_new: each bit is independently 1 with probability
↳ (1-s)
np.random.seed(42)
X_list = []
for s in sparsities:
    pattern = (np.random.rand(N) > s) # True 1, with probability (1-s)
    X_list.append(pattern)
patterns = np.array(X_list)

```

```

[ ]: config = setup_write_experiment(
    name="pattern_push_heterogeneous",
    patterns=patterns,
    params={"leak": 1.0, "drive_target": 6.0, "learning_rate": 0.0001,
           "momentum": 0.9, "max_iterations": 5000, "convergence_threshold": 0.
↳ 01}
)
run_cpp("write", config)

```

Running: /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/bin/write /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/data/configs/pattern_push_heterogeneous/config.json

Loaded 5 patterns of size 600

Running 1 training simulations

Training sim 0

Sim 0 converged in 97 iterations

All simulation data has been written to all_simulation_data.csv

Training complete. Results in: /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/data/trained_networks/pattern_push_heterogeneous

```

[ ]: CompletedProcess(args=['/mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/bin/write', '/mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/data/configs/pattern_push_heterogeneous/config.json'], returncode=0)

```

```

[ ]: sim_dir = DATA_DIR / "trained_networks" / "pattern_push_heterogeneous" /
↳ "sim_nb_0"
W = np.loadtxt(sim_dir / "weights.data")
X = np.loadtxt(sim_dir / "patterns.data")
v_N = np.ones(N) * 0.5

# Network derivative at neutral state
sigma = lambda x: 1.0 / (1.0 + np.exp(-x))

```

```

f = -v_N + sigma(W @ v_N)

# Compute push P_mu for each pattern
P = np.array([
    np.dot(f, (X[mu] - v_N) / np.linalg.norm(X[mu] - v_N))
    for mu in range(M)
])

print("Push values at neutral state (heterogeneous sparsities):")
for mu in range(M):
    print(f"  Pattern {mu} (s={sparsities[mu]:.2f}): P_{mu} = {P[mu]:.4f}")

```

Push values at neutral state (heterogeneous sparsities):

```

Pattern 0 (s=0.40): P_0 = 4.1754
Pattern 1 (s=0.45): P_1 = 3.7924
Pattern 2 (s=0.50): P_2 = 3.2664
Pattern 3 (s=0.55): P_3 = 3.0122
Pattern 4 (s=0.60): P_4 = 2.5921

```

```

[ ]: fig, ax = plt.subplots(figsize=(8, 4))

bars = ax.bar(range(M), P, facecolor='white', edgecolor='black', linewidth=2.5,
    width=0.6)
ax.axhline(0, color='black', linewidth=1.5)

# Add sparsity annotations above each bar
for mu, (bar, s) in enumerate(zip(bars, sparsities)):
    ax.annotate(f'$s$={s:.2f}', xy=(bar.get_x() + bar.get_width()/2, bar.
        get_height()),
        xytext=(0, 5), textcoords='offset points', ha='center',
        va='bottom', fontsize=25)

# Expand y-axis to fit annotations
ymin, ymax = ax.get_ylim()
ax.set_ylim(ymin, ymax + 0.15 * (ymax - ymin))

ax.set_xlabel(r'Pattern index $\mu$')
ax.set_ylabel(r'Push $P_\mu$')
ax.set_xticks(range(M))
ax.tick_params(axis='both', which='both', bottom=True, left=True, top=False,
    right=False)

for spine in ['bottom', 'left']:
    ax.spines[spine].set_visible(True)
    ax.spines[spine].set_color('black')
    ax.spines[spine].set_linewidth(1.5)
for spine in ['top', 'right']:

```

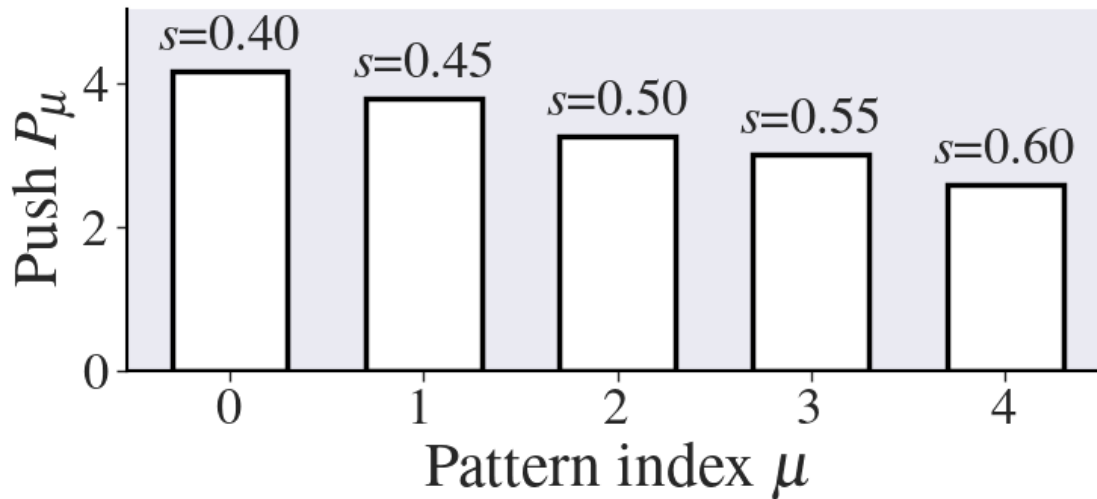
```

ax.spines[spine].set_visible(False)

plt.tight_layout()
plt.savefig(PLOTS_DIR / 'pattern_push_heterogeneous.png', dpi=300,
    ↳bbox_inches='tight')
print(f"Saved to {PLOTS_DIR / 'pattern_push_heterogeneous.png'}")
plt.show()

```

Saved to /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/scripts/plots/pattern_push_heterogeneous.png



```

[ ]: sleep_config = setup_sleep_experiment(
    name="pattern_push_heterogeneous",
    trained_networks_dir=DATA_DIR / "trained_networks" /
    ↳"pattern_push_heterogeneous",
    params={
        "beta": 0.1, "delta": 0.01, "max_queries": M,
        "stop_on_spurious": 0, "stop_on_all_found": 0,
        "save_inhibition_matrices": 1
    }
)
run_cpp("sleep", sleep_config)

```

Running: /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/bin/sleep /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/data/configs/pattern_push_heterogeneous/config.json

Found 1 trained networks

Running 1 sleep simulations

Sleep sim 0

Sim 0: found 4/5 patterns, 0 spurious

Extracted Sim ID 0

All simulation data has been written to all_simulation_data.csv

Final results have been written to final_results.csv (1 simulations)

Sleep simulations complete. Results in: /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/data/sleep_results/pattern_push_heterogeneous

```
[ ]: CompletedProcess(args=['/mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/bin/sleep', '/mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/data/configs/pattern_push_heterogeneous/config.json'], returncode=0)
```

```
[ ]: sleep_dir = DATA_DIR / "sleep_results" / "pattern_push_heterogeneous" /   
      ↪ "sim_nb_0"  
      results = pd.read_csv(sleep_dir / "results.data")  
      recovered_indices = results["recovered_pattern_idx"].values  
  
      num_iters = len(results)  
      P_evolution = np.zeros((num_iters, M))  
  
      for t in range(num_iters):  
          A = np.loadtxt(sleep_dir / f"inhib_matrix_iter_{t}.data")  
          A_diag = np.diag(A)  
          f_t = -v_N + sigma(W @ v_N) - A_diag * v_N  
          P_evolution[t] = [np.dot(f_t, (X[mu] - v_N) / np.linalg.norm(X[mu] - v_N))]  
          ↪ for mu in range(M)]
```

```
[ ]: fig, axes = plt.subplots(num_iters, 1, figsize=(6, 2.5*num_iters), sharex=True,   
      ↪ sharey=True)  
      axes = np.atleast_1d(axes)  
  
      explored_set = set()  
      for t in range(num_iters):  
          ax = axes[t]  
          colors = ['white'] * M  
  
          for idx in explored_set:  
              colors[idx] = '#FFD699' # Light orange (past explored)  
  
          if recovered_indices[t] >= 0:  
              colors[recovered_indices[t]] = '#d62728' # Red (just retrieved)  
              explored_set.add(recovered_indices[t])  
  
          bars = ax.bar(range(M), P_evolution[t], color=colors, edgecolor='black',   
          ↪ linewidth=2, width=0.6)  
          ax.axhline(0, color='black', linewidth=1)
```

```

ax.set_ylabel(f'Iter {t}')

# Add sparsity annotations above each bar (only on first subplot)
if t == 0:
    for mu, (bar, s) in enumerate(zip(bars, sparsities)):
        ax.annotate(f'$s$={s:.2f}', xy=(bar.get_x() + bar.get_width()/2,
        ↪ bar.get_height()),
                    xytext=(0, 5), textcoords='offset points', ha='center',
        ↪ va='bottom', fontsize=20)

    for spine in ['bottom', 'left']:
        ax.spines[spine].set_visible(True)
        ax.spines[spine].set_color('black')
        ax.spines[spine].set_linewidth(1.5)
    for spine in ['top', 'right']:
        ax.spines[spine].set_visible(False)

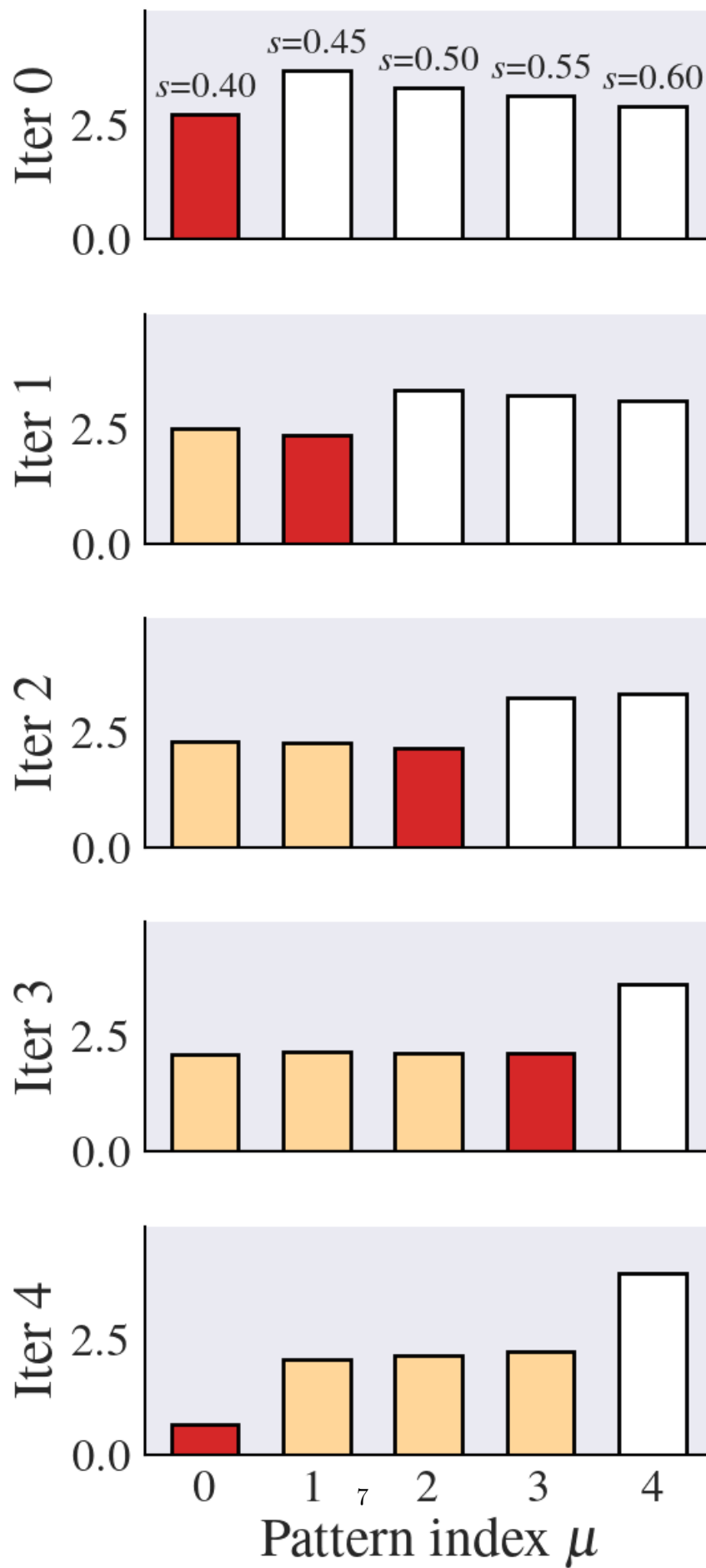
axes[-1].set_xlabel(r'Pattern index $\mu$')
axes[-1].set_xticks(range(M))

# Expand y-axis to fit annotations on first subplot
ymin, ymax = axes[0].get_ylim()
axes[0].set_ylim(ymin, ymax + 0.2 * (ymax - ymin))

plt.tight_layout()
plt.savefig(PLOTS_DIR / 'pattern_push_heterogeneous_AR_evolution.png', dpi=300,
        ↪ bbox_inches='tight')
print(f"Saved to {PLOTS_DIR / 'pattern_push_heterogeneous_AR_evolution.png'}")
plt.show()

```

Saved to /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN/simulations/main_write_sleep/scripts/plots/pattern_push_heterogeneous_AR_evolution.png



Statistical Analysis: Visit Count and First Visit Order vs Sparsity

This section runs multiple independent simulations to analyze how pattern sparsity affects retrieval dynamics during Autonomous Retrieval (AR).

Metrics Definition

1. Visit Count For each pattern μ , the **visit count** is the total number of times the network converged to that pattern during AR. For a simulation with T retrieval attempts:

$$\text{visit_count}_\mu = \sum_{t=1}^T \mathbb{I}[\text{retrieved}_t = \mu]$$

2. First Visit Iteration The **first visit iteration** is the index of the AR iteration at which pattern μ was first retrieved. If pattern μ is never retrieved, we assign a value of $T_{\max} + 1$ (excluded from the box plot analysis).

$$\text{first_visit}_\mu = \min\{t : \text{retrieved}_t = \mu\}$$

Experimental Protocol

For each load $p \in \{4, 6, 8\}$: 1. Generate p patterns with linearly spaced sparsities in $[0.3, 0.7]$ 2. Train a network of size $N = 600$ on these patterns 3. Run AR from the neutral state with inhibitory plasticity ($\beta = 0.05$) 4. Record which pattern is retrieved at each iteration 5. Repeat for 30 independent simulations (different random seeds)

Patterns are grouped into 5 sparsity bins for visualization.

Box Plot Interpretation

Each box plot displays the distribution of a metric across simulations: - **Red line**: Median (50th percentile) - **Box**: Interquartile range (IQR), from Q1 (25th percentile) to Q3 (75th percentile) - **Whiskers**: Extend to the most extreme data points within $1.5 \times \text{IQR}$ - **Black dots**: Outliers, i.e., data points beyond the whiskers

Key Observation

Less sparse patterns (lower s , more active neurons) are visited more frequently and discovered earlier during AR. This is consistent with the push analysis above: patterns with more active neurons have a larger projection of the network derivative onto their direction, resulting in stronger attraction from the neutral state.

```
[ ]: from utils import list_simulations, build
    from tqdm import tqdm

LOADS = [4, 6, 8]
NB_REPETITIONS = 30
SPARSITY_MIN = 0.3
SPARSITY_MAX = 0.7
MAX_QUERIES_STATS = 200
```



```
N = 200
EXPERIMENT_NAME_STATS = "pattern_push_heterogeneous_stats"
```

```
[ ]: build(verbose=False)
```

```
[ ]: True
```

```
[ ]: total_train = len(LOADS) * NB_REPETITIONS
with tqdm(total=total_train, desc="Training", unit="net") as pbar:
    for p in LOADS:
        for rep in range(NB_REPETITIONS):
            pbar.set_postfix_str(f"p={p}, rep={rep}")

            # Generate heterogeneous patterns for this repetition
            np.random.seed(rep * 1000 + p * 100)
            sparsities_rep = np.linspace(SPARSITY_MIN, SPARSITY_MAX, p)
            patterns_rep = np.array([(np.random.rand(N) > s) for s in
↪sparsities_rep])

            config = setup_write_experiment(
                name=f"{EXPERIMENT_NAME_STATS}_p{p}_rep{rep}",
                patterns=patterns_rep,
                params={"leak": 1.0, "drive_target": 6.0, "learning_rate": 0.
↪0001,
                        "momentum": 0.9, "max_iterations": 5000,
↪"convergence_threshold": 0.01}
            )
            run_cpp("write", config, verbose=False)
            pbar.update(1)
```

```
Training: 100%|          | 90/90 [00:25<00:00, 3.59net/s, p=8, rep=29]
```

```
[ ]: total_sleep = len(LOADS) * NB_REPETITIONS
with tqdm(total=total_sleep, desc="Sleep", unit="sim") as pbar:
    for p in LOADS:
        for rep in range(NB_REPETITIONS):
            pbar.set_postfix_str(f"p={p}, rep={rep}")

            trained_dir = DATA_DIR / "trained_networks" /
↪f"{EXPERIMENT_NAME_STATS}_p{p}_rep{rep}"

            sleep_config = setup_sleep_experiment(
                name=f"{EXPERIMENT_NAME_STATS}_p{p}_rep{rep}",
                trained_networks_dir=trained_dir,
                params={
                    "beta": 0.05, "delta": 0.01, "max_queries":
↪MAX_QUERIES_STATS,
```

```

        "noise_dynamics": 1, "stddev_dynamics": 0.01,
        "stop_on_spurious": 1, "stop_on_all_found": 1,
    }
)
run_cpp("sleep", sleep_config, verbose=False)
pbar.update(1)

```

Sleep: 100%| | 90/90 [02:41<00:00, 1.80s/sim, p=8, rep=29]

```

[ ]: all_data = []

for p in LOADS:
    sparsities_p = np.linspace(SPARSITY_MIN, SPARSITY_MAX, p)

    for rep in range(NB_REPETITIONS):
        sleep_dir = DATA_DIR / "sleep_results" /
        ↪f"{EXPERIMENT_NAME_STATS}_p{p}_rep{rep}" / "sim_nb_0"
        results_file = sleep_dir / "results.data"

        if not results_file.exists():
            print(f" Warning: No results for p={p}, rep={rep}")
            continue

        results_df = pd.read_csv(results_file)

        # Compute visit counts and first visit order
        visit_counts = {mu: 0 for mu in range(p)}
        first_visit = {mu: None for mu in range(p)}

        for iter_idx, row in results_df.iterrows():
            pattern_idx = int(row['recovered_pattern_idx'])
            if 0 <= pattern_idx < p:
                visit_counts[pattern_idx] += 1
                if first_visit[pattern_idx] is None:
                    first_visit[pattern_idx] = iter_idx

        # Store data for each pattern
        for mu in range(p):
            all_data.append({
                "load": p,
                "repetition": rep,
                "pattern_idx": mu,
                "sparsity": sparsities_p[mu],
                "visit_count": visit_counts[mu],
                "first_visit": first_visit[mu] if first_visit[mu] is not None
            ↪else MAX_QUERIES_STATS + 1
            })

```

```
df_stats = pd.DataFrame(all_data)
```

```
[ ]: n_bins = 5
bin_edges = np.linspace(SPARSITY_MIN, SPARSITY_MAX, n_bins + 1)
bin_centers = [(bin_edges[i] + bin_edges[i+1])/2 for i in range(n_bins)]
bin_labels = [f"{c:.2f}" for c in bin_centers]

df_stats["sparsity_bin"] = pd.cut(df_stats["sparsity"], bins=bin_edges,
    ↳ labels=bin_labels, include_lowest=True)

fig, axes = plt.subplots(1, len(LOADS), figsize=(6*len(LOADS), 6), sharey=True)
axes = np.atleast_1d(axes)

for i, p in enumerate(LOADS):
    ax = axes[i]
    df_p = df_stats[df_stats["load"] == p].dropna(subset=["sparsity_bin"])

    box_data = []
    valid_labels = []
    for label in bin_labels:
        data = df_p[df_p["sparsity_bin"] == label]["visit_count"].values
        if len(data) > 0:
            box_data.append(data)
            valid_labels.append(label)

    if not box_data:
        continue

    bp = ax.boxplot(box_data, patch_artist=True, widths=0.6)

    for box in bp['boxes']:
        box.set_facecolor('white')
        box.set_edgecolor('black')
        box.set_linewidth(2)
    for whisker in bp['whiskers']:
        whisker.set_color('black')
        whisker.set_linewidth(2)
    for cap in bp['caps']:
        cap.set_color('black')
        cap.set_linewidth(2)
    for median in bp['medians']:
        median.set_color('#d62728')
        median.set_linewidth(2.5)
    for flier in bp['fliers']:
        flier.set_markerfacecolor('black')
        flier.set_markeredgcolor('black')
```

```

flier.set_markersize(5)

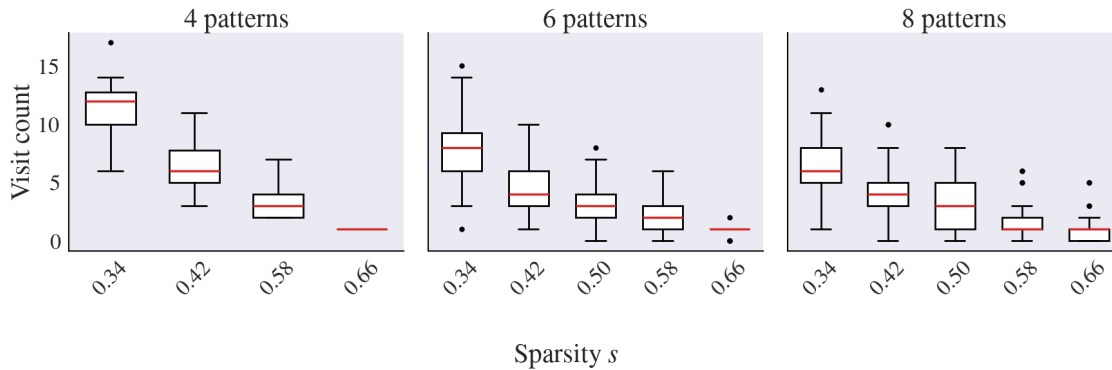
ax.set_xticklabels(valid_labels, rotation=45)
ax.set_title(rf'{p} patterns')

if i == 0:
    ax.set_ylabel(r'Visit count')

for spine in ['bottom', 'left']:
    ax.spines[spine].set_visible(True)
    ax.spines[spine].set_color('black')
    ax.spines[spine].set_linewidth(1.5)
for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)

fig.supxlabel(r'Sparsity $s$')
plt.tight_layout()
output_path = PLOTS_DIR / 'pattern_push_heterogeneous_visit_count.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.show()

```



```

[ ]: df_visited = df_stats[df_stats["first_visit"] <= MAX_QUERIES_STATS].copy()
df_visited["sparsity_bin"] = pd.cut(df_visited["sparsity"], bins=bin_edges,
    ↪ labels=bin_labels, include_lowest=True)

fig, axes = plt.subplots(1, len(LOADS), figsize=(6*len(LOADS), 6), sharey=True)
axes = np.atleast_1d(axes)

for i, p in enumerate(LOADS):
    ax = axes[i]
    df_p = df_visited[df_visited["load"] == p].dropna(subset=["sparsity_bin"])

    box_data = []

```

```

valid_labels = []
for label in bin_labels:
    data = df_p[df_p["sparsity_bin"] == label]["first_visit"].values
    if len(data) > 0:
        box_data.append(data)
        valid_labels.append(label)

if not box_data:
    continue

bp = ax.boxplot(box_data, patch_artist=True, widths=0.6)

for box in bp['boxes']:
    box.set_facecolor('white')
    box.set_edgecolor('black')
    box.set_linewidth(2)
for whisker in bp['whiskers']:
    whisker.set_color('black')
    whisker.set_linewidth(2)
for cap in bp['caps']:
    cap.set_color('black')
    cap.set_linewidth(2)
for median in bp['medians']:
    median.set_color('#d62728')
    median.set_linewidth(2.5)
for flier in bp['fliers']:
    flier.set_markerfacecolor('black')
    flier.set_markeredgcolor('black')
    flier.set_markersize(5)

ax.set_xticklabels(valid_labels, rotation=45)
ax.set_title(rf'{p} patterns')

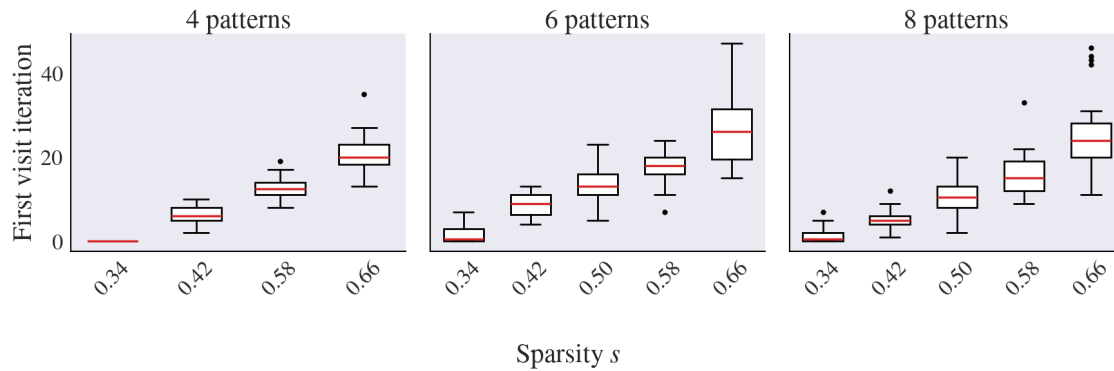
if i == 0:
    ax.set_ylabel(r'First visit iteration')

for spine in ['bottom', 'left']:
    ax.spines[spine].set_visible(True)
    ax.spines[spine].set_color('black')
    ax.spines[spine].set_linewidth(1.5)
for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)

fig.supxlabel(r'Sparsity $s$')
plt.tight_layout()
output_path = PLOTS_DIR / 'pattern_push_heterogeneous_first_visit.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')

```

```
plt.show()
```



```
[ ]: print("\nCorrelations (sparsity vs metric):")
for p in LOADS:
    df_p = df_stats[df_stats["load"] == p]
    corr_visit = df_p["sparsity"].corr(df_p["visit_count"])
    df_p_visited = df_visited[df_visited["load"] == p]
    corr_first = df_p_visited["sparsity"].corr(df_p_visited["first_visit"]) if
    len(df_p_visited) > 0 else float('nan')
    print(f"  p={p}: visit_count r={corr_visit:.3f}, first_visit r={corr_first:.
    3f}")
```

```
Correlations (sparsity vs metric):
p=4: visit_count r=-0.916, first_visit r=0.943
p=6: visit_count r=-0.812, first_visit r=0.883
p=8: visit_count r=-0.728, first_visit r=0.878
```